

Privacy-Preserving Compliance

by

Joss Duff

A Thesis

Presented to the Graduate and Research Committee
of Lehigh University
in Candidacy for the Degree of
Master of Science

in
Computer Science

Lehigh University

May 2026

PREVIEW

© 2026 Copyright
Joss Duff

Thesis Approval Signature Sheet

Thesis is accepted and approved in partial fulfillment of the requirements for the Master of Science in Computer Science.

Privacy-Preserving Compliance
Joss Duff

Date Approved

Henry F. Korth, Thesis Advisor

Brian Davidson, Department Chair

Acknowledgments

I am forever grateful to Lehigh Computer Science and Engineering for providing me the resources and opportunity to pursue my interests. Lehigh's Scalable Systems and Software research group and Lehigh Blockchain gave me a supportive environment to discuss ideas and make friends. I am lucky to be surrounded by such bright peers.

Most importantly, I would like to thank Professor Korth for his continued mentorship. Professor Korth introduced me to the field of blockchain during my time as an undergraduate at Lehigh and continued to foster my interest through my Master's.

I also thank my friends Jimjim Valkema, Tal Derei, Tim Clancy, and Tom Lehman for feedback and discussion throughout the development of this thesis. Lehigh Blockchain's Kevin Cho, Max Nussbaum, and Louis Scalise provided feedback on this written work and code implementation.

Finally, I thank my friends and family for always supporting my pursuits.

Table of Contents

Acknowledgments	iv
Abstract	1
1 Background and Definitions	2
1.1 Zero-Knowledge Proof	2
1.2 Compliance	3
1.3 Blockchain Background	3
1.4 Privacy in the Blockchain Setting	5
2 Problem	8
2.1 Industry Interest	9
3 Desired Privacy-Preserving Compliance Properties	10
3.1 No Deanonimization	10
3.2 Proactive Compliance	11
3.3 Rich Compliance Language	11
3.4 Multiple Compliance	12
3.5 Secret Inputs	12
3.6 Verifiable Compliance	12
3.7 Chain Agnostic	13
3.8 Modular Privacy	13
4 Related Work and Attempted Solutions	14
5 Proposed Solution	17
5.1 Framework Approach	17
5.2 Actors	18
5.3 Composability	20
5.4 Constraints	22
5.5 Compliance Definition Contract	27
5.6 Generating Compliance Proofs	28
5.7 Proof Manager	29
5.8 Example Integrating Application: Compliant Stablecoin	30
5.9 Updating Compliance Definitions	31
5.10 Security Considerations	33
6 Prototype Implementation	36
6.1 Prototype Demo	36
6.2 Benchmarks	37
6.3 MEM Constraint	38

6.4	NON-MEM Constraint	38
7	Future Work	40
7.1	Constraint Domain-Specific Language	40
7.2	Proof Inference and Optimization	41
7.3	Backend Provers	42
7.4	Carrying Compliance Proof Through Mixers	42
7.5	Alternative Applications of Constraint Proving	44
7.6	Latency Optimization	44
7.7	Industrial and Regulator Acceptance	44
	Bibliography	45
A	Appendix: Framework Specification	52
A.1	Terminology	52
A.2	On-Chain Contracts	52
A.3	Proof System Requirements	57
A.4	Off-Chain Artifact Storage	59
A.5	Interaction Protocols	61
B	Appendix: Example Constraints	65
B.1	Non-Membership Constraint (NON-MEM)	65
B.2	Membership Constraint (MEM)	65
B.3	Protocol Interaction Constraint (INTERACT)	66
B.4	Protocol Avoidance Constraint (AVOID)	66
B.5	Age Constraint (AGE)	66
B.6	Structuring Constraint (STRUCTURE)	66
	Vita	68

List of Tables

4.1	Comparison of privacy-preserving compliance solutions	16
6.1	MEM constraint benchmarks	38
6.2	NON-MEM constraint benchmarks	39

PREVIEW

List of Figures

5.1	Composable compliance framework overview	21
5.2	Multi-compliance example	21
5.3	Proof aggregation workflow	28
5.4	Proof Manager architecture	30

PREVIEW

Abstract

Privacy and compliance in blockchain systems face an inherent contradiction: privacy requires that only the individual knows certain information, while compliance traditionally requires revealing information to prove compliance criteria. This tension has led to two unsatisfactory outcomes:

1. Privacy systems without compliance attract illicit actors and face legal challenges.
2. Compliance systems without privacy expose all transaction histories publicly, making them unacceptable for consumer and commercial adoption.

Neither outcome supports the widespread adoption of blockchain technology, particularly among traditional financial institutions where regulatory uncertainty and privacy concerns are the top barriers to entry.

We present a composable framework that resolves this conflict by enabling users to prove compliance cryptographically without revealing private transaction data. In our system, regulatory bodies publish compliance definitions as combinations of reusable logical primitives called constraints. Applications select relevant compliance definitions based on their regulatory obligations. Users generate zero-knowledge proofs demonstrating that they meet these requirements, which are then verified on-chain before transactions execute. This provides proactive compliance: preventing non-compliant actions rather than detecting them afterward, all while maintaining strong privacy guarantees.

Our framework is distinguished by its composability and generality, designed explicitly for a global ecosystem with many regulatory bodies whose requirements evolve over time. Compliance definitions are chain-agnostic, modular with respect to privacy mechanisms, versioned to accommodate regulatory updates, and can be combined arbitrarily which allows applications to satisfy multiple jurisdictions simultaneously.

1. Background and Definitions

Below is relevant background information on blockchains, blockchain privacy systems, ZK proofs, and our working definition of “compliance”. People familiar with blockchain and ZK can skip this section.

1.1 Zero-Knowledge Proof

Zero-knowledge (ZK) proofs are a cryptographic technique that allows a party to prove specific claims about a computation without revealing any additional information about the computation. The origins of this concept predate blockchain [7, 26], but it was described elegantly and simply by Vitalik Buterin et al. in [13] as follows:

The “claim” proven by a ZK proof is expressed as a type of program that is often called a *circuit*. Mathematically, it suffices to think of it as a function $f(x, w) \rightarrow \{\text{True}, \text{False}\}$, where x is the public input, w is the private input, and $f(\cdot)$ is the function being computed. A ZK proof proves that, for a given x known both by the prover and the verifier, the prover knows a w such that $f(x, w)$ returns True.

SNARKs are a specific type of ZK proof that has the unique property of *succinctness*: The proof size does not grow with the size of the computation, and the proof can be verified very quickly relative to the computation [45].¹

A drawback with some SNARK schemes, such as Groth16 [27], is that they rely on a *trusted setup*: a time-intensive ceremony involving multiple participants. A trusted setup needs to be performed once per circuit [34] for Groth16 and some other SNARKs.

¹The SNARK acronym comes from Succinct Non-interactive ARGuments of Knowledge. The non-interactive term serves to distinguish SNARKs from the interactive proofs between a prover and a verifier that are often used to illustrate the concept of ZK for a general audience.

A type of SNARK proofs, PLONK, mitigates this drawback by allowing trusted setups to be re-used across circuits. With PLONK proofs, it is not necessary to initiate a new trusted setup for every new circuit [25].

1.2 Compliance

Compliance is referred to as the goals and properties of a system to attempt to adhere to a law or policy. Compliance is not immunity from prosecution, it is a best effort to prove legality of actions. For example, US companies have systems in place to prevent money laundering in order to be compliant with the US Bank Secrecy Act (BSA) - also referred to as Anti-Money Laundering (AML) [61]. A typical effort to comply with AML in the blockchain space is blocking addresses associated with sanctioned individuals from interacting with a protocol.

A goal of this framework is to be flexible enough to accommodate compliance definitions over all aspects of a transaction: the payer, the payee, the content of the transaction, and even the actors' transaction histories. Flexibility results in the ability to express any definition of compliance. This framework is built around the fact that definitions of compliance vary over time and by jurisdiction.

1.3 Blockchain Background

Blockchains are fundamentally data-storage systems with unique properties. Most importantly, data stored on a blockchain (on-chain) are immutable: history cannot be changed. This property is achieved by allowing anyone around the world to participate in consensus and validation of the blockchain's state by running a node. There is a small class of *permissioned* blockchains with access restrictions on who can use the chain or run a node.

Nodes have a financial incentive to participate correctly in consensus. This mechanism makes it economically infeasible for a single attacker to disrupt the network. For example, in the Ethereum network, nodes who want to participate in block production must put up a stake of the Ether currency² which is “*slashed*” (lost forever) if the node behaves incorrectly[52].

²The original staking requirement was 32 Ether but it is subject to change.

The first blockchain, Bitcoin, was created in 2008 and proposed a trustless ledger of transaction histories[40]. Ethereum was created in 2013 and improved on Bitcoin by allowing trustless execution of Turing-complete logic on-chain, referred to as *smart contracts* [11]. This allowed for more complex applications like marketplaces and lending platforms to exist in a decentralized, permissionless, and immutable setting. These on-chain applications are sometimes referred to as “decentralized applications” or *dapps* for short.

Every transaction on a blockchain has a non-static monetary cost paid to the block-producing node as an incentive to include the transaction in the next write. Some blockchains with smart-contract support, like Ethereum, include an additional transaction cost driven by the complexity of the smart-contract logic being executed.

Users on blockchains are represented by a public-key address. The user signs messages and transactions with the associated private key. Users can have multiple public-key addresses. It is impossible to reveal who is associated with a public key unless the user’s identity is revealed. Identity can be revealed directly by the user or indirectly by others who know the public key’s identity, like a centralized exchange.

A centralized exchange (CEX) is a company that allows users to trade traditional currency for cryptocurrency. They often require the user to connect their bank account and supply regulatory documentation such as identification cards and proof of residency. They collect this information to comply with “Know Your Customer” (KYC) laws. Using a CEX is a common method for acquiring cryptocurrency. CEXs are *custodial*: they control the address (they know the private key) that owns the cryptocurrency on-chain and allow the user the ability to trade or transfer the cryptocurrency. They also allow users to send their cryptocurrency from the CEX account to the users’ own on-chain address. CEX accounts are not addresses. Rather, the CEX keeps its own addresses and the user’s holdings are only an accounting record in the CEX’s books.

A decentralized exchange (DEX) enables trading cryptocurrency while letting users retain direct control of their holdings. In its simplest form, the DEX creates a smart-contract to implement the trade and the users sign it with their private key.

1.4 Privacy in the Blockchain Setting

All data on blockchains are public by default unless the chain supports special privacy features like Aztec Network [66] or Zcash [8]. There are also some enterprise permissioned blockchains that restrict visibility levels. However, the vast majority of chains are public by default, including Ethereum and Bitcoin.

For public blockchains, every user’s entire transaction history is exposed to the world. This is obviously undesirable for consumers. Most people do not want their purchase or subscription history visible to the entire world. Many applications and protocols exist to provide users varying degrees of privacy.

Blockchain privacy protocols fall into two broad categories:

- *App-layer privacy*: Applications on top of existing blockchains.
- *Base-layer privacy*: Entirely new blockchain architectures with privacy as a core identity.

1.4.1 App-layer Privacy

CoinJoin was the first permissionless app-layer transaction mixer [36]. CoinJoin was built on Bitcoin and combined multiple input Unspent Transaction Outputs (UTXOs) from different parties into multiple outputs, obscuring which inputs correspond to which outputs. Tornado Cash was the first transaction mixer to use ZK proofs [44], and saw massive popularity [21]. ZK proofs are a common tool used to achieve privacy; ZK allows a party to prove a computation without revealing the entire computation. See Section 1.1 for an introduction to ZK proofs. The black-hat group Lazarus used Tornado Cash to aid in laundering stolen funds and the Tornado Cash developers were put on trial [62]. Railgun [50] and Hinkal [28] are transaction mixers that focus on compliance. Privacy Pools is a recent transaction mixer that also focuses on compliance with *association sets* [13]. Ultra-Anon is an experimental privacy token that has the property of *plausible deniability*: it isn’t revealed if users utilized privacy [23]. Stealth addresses are ephemeral public keys that can receive any tokens while hiding the receiver address [12]. zkBob is a wallet that has private transfer for certain stablecoins [68]. Webb Protocol creates a cross-chain anonymity set [57]. Worm

is an application implementation of “EIP-7503: Zero-Knowledge Wormholes” [32], which is another transactional privacy solution with the property of plausible deniability.

1.4.2 Base-layer Privacy

Zcash is a privacy blockchain based on Bitcoin’s UTXO model. Zcash is implemented from the 2014 Zerocash paper [8]. Monero is another private blockchain that leverages the UTXO system and is based on the 2013 CryptoNote paper [63]. Grin is an implementation of the 2016 Mimblewimble protocol, a modification of Bitcoin with private transfers [29]. Aztec is a general purpose layer 2³ blockchain that allows for private state with smart-contract support [66]. Aleo [2], based on the 2018 paper “ZEXE: Enabling Decentralized Private Computation” [9], is also a blockchain with private state and smart contracts. The Miden blockchain supports the private state similarly [47]. Penumbra is a private network and decentralized exchange for the Cosmos ecosystem [43]. Arc [35] and Tempo [58], founded by Circle and Stripe, respectively, are stablecoin-focused blockchains with some degree of transaction privacy. Payy is an Ethereum layer 2 with native transaction privacy and compliance mechanisms [37]. Secret Network has private by default smart contract state [54]. Ernst & Young (EY) released a permissioned layer 2 blockchain, Nightfall, with private transactions and compliance features [24].

1.4.3 Custodial Mixers

Custodial mixers are transaction privacy solutions that require users to send funds to a trusted intermediary who then redistributes the funds. This accomplishes the privacy goal of breaking the on-chain traceability between the input and output transactions, but custodial mixers have the major weakness that operators could steal funds or keep records. This is fundamentally weaker privacy because that intermediary has knowledge of a user’s transaction history, rather than that history being private and held only by the user. While this is not a legal document, we note that in some jurisdictions, the intermediary could be subject to government subpoena.

Blender.io was an example of a custodial mixer on Bitcoin where users sent funds to a

³A layer 2 blockchain periodically syncs its state to another blockchain, referred to as the layer 1. In doing so, the layer 2 blockchain inherits the security and data guarantees of the layer 1.

trusted operator who then re-distributed the funds. Blender was sanctioned in 2022 by the U.S. Treasury Department for failing to meet AML regulations [59].

Some centralized exchanges can also be used as custodial mixers. A user deposits into the centralized exchange from one address and withdraws from another address. From the on-chain perspective, this looks like two unrelated transactions among the millions that occur within that centralized exchange. However, the centralized exchange has full information and knows the two addresses are owned by the same user.

1.4.4 Anonymity Set

An *anonymity set* is the group of entities among which an observer cannot reliably identify which specific entity possesses a particular attribute or performed a particular action [46, 49]. Anonymity scales with set size: the larger the anonymity set, the less identifiable each entity is [22].

For transaction mixer applications on blockchains like Railgun or Tornado Cash, any address that has deposited can withdraw, but the link between depositing and withdrawing addresses is never revealed. Each withdrawal could have come from any of the depositing addresses. Therefore, the anonymity set of any given withdrawal from the mixer is the set of all addresses that have deposited in the mixer. This description is oversimplified, and in practice anonymity sets can be reduced with chain analysis [31, 38]. Anonymity here is referring to the obfuscation of the link between a depositing and withdrawing address.

2. Problem

Blockchains are fundamentally pseudonymous and public: user identities are obfuscated behind addresses, but all activity between addresses is public by default. In practice, entities are able to associate identities to addresses when on-ramping traditional funds to cryptocurrency and therefore track individuals' flows of funds on-chain. In order for cryptocurrency to be practical for consumers, it must at the very least have the same level of privacy as traditional finance, but ideally improves on traditional finance privacy. In traditional finance, typically two entities know information about a user's funds; the user and the exchange that is custodian of the funds. In blockchain settings, all entities can track user funds and activity. Neither consumers nor corporations will want their entire trading, purchasing, or subscription histories publicly available.

Privacy is a growing field in the blockchain space [44, 32, 8, 23], but raises the concern that malicious actors may use privacy to launder funds. In an effort to reduce money laundering, regulatory bodies are pursuing legal action against developers of privacy protocols [33, 62, 18]. This introduces the need to explicitly meet regulatory compliance inside private on-chain applications.

The goal is to maximize privacy while maintaining the ability to prove compliance. Current research efforts attempting to bridge privacy and compliance have proposed solutions with strict domain limitations such as a new blockchain architecture, specific on-chain protocols, or privacy violations. We investigate a framework that allows for compliance in any blockchain smart-contract environment and in any privacy protocol without compromising privacy provided by the protocol.